COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	07/30/2026	**DATE SUBMITTED:**	07/30/2026
NAME:	Charles Vence Espinas	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **SSH Key-Based Authentication and GIT Setup**

1. PROCEDURE OUTPUT

Task 1: Create an SSH Key Pair for User Authentication

1. The simplest way to generate a key pair is to run ssh-keygen without arguments. In this case, it will prompt for the file in which to store keys. First, the tool asked where to save the file. SSH keys for user authentication are usually stored in the users .ssh directory under the home directory. However, in enterprise environments, the location is often different. The default key file name depends on the algorithm, in this case id_rsa when using the default RSA algorithm. It could also be, for example, id_dsa or id_ecdsa.

2. Issue the command ssh-keygen -t rsa -b 4096. The algorithm is selected using the -t option and key size using the -b option.

3. When asked for a passphrase, just press enter. The passphrase is used for encrypting the key, so that it cannot be used even if someone obtains the private key file. The passphrase should be cryptographically strong.

```
charles-vence-espinas@workstation:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/charles-vence-espinas/.ssh/id_rsa):
/home/charles-vence-espinas/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/charles-vence-espinas/.ssh/id_rsa
Your public key has been saved in /home/charles-vence-espinas/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:CQrhk8xfX6xZjNSxVRqevTdgbXc6kctd+OJHaKGVQc4 charles-vence-espinas@workstation
The key's randomart image is:
+---[RSA 4096]----+
|  .     ....oo+  |
| + o   . +.+ B.= |
|  B   o . * +oEo+|
|   + o o *  .+oX+|
|    o   S   . X.=|
|             o =.|
|              . .|
|               . |
|                 |
+----[SHA256]-----+
```

4. Verify that you have created the key by issuing the command ls -la .ssh. The command should show the .ssh directory containing a pair of keys. For example, id_rsa.pub and id_rsa.

```
charles-vence-espinas@workstation:~$ ls -la .ssh
total 28
drwx------  3 charles-vence-espinas charles-vence-espinas 4096 Jul 23 18:45 .
drwxr-x--- 17 charles-vence-espinas charles-vence-espinas 4096 Jul 28 23:05 ..
-rw-------  1 charles-vence-espinas charles-vence-espinas    0 Mar 23 22:13 authorized_key
s
-rw-------  1 charles-vence-espinas charles-vence-espinas 3401 Jul 30 21:20 id_rsa
-rw-r--r--  1 charles-vence-espinas charles-vence-espinas  759 Jul 30 21:20 id_rsa.pub
-rw-------  1 charles-vence-espinas charles-vence-espinas 1688 Jul 30 21:14 known_hosts
-rw-------  1 charles-vence-espinas charles-vence-espinas  284 Jul 23 18:45 known_hosts.ol
d
drwx------  2 charles-vence-espinas charles-vence-espinas 4096 Jul 23 18:32 ssh-copy-id.JO
yzSJ5hQ3
```

Task 2: Copying the Public Key to the remote servers

1. To use public key authentication, the public key must be copied to a server and installed in an authorized_keys file. This can be conveniently done using the ssh-copy-id tool.

2. Issue the command similar to this: ssh-copy-id -i ~/.ssh/id_rsa user@host

```
charles-vence-espinas@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa charles-vence-espinas@server1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/charles-vence-espinas/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
charles-vence-espinas@server1's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'charles-vence-espinas@server1'"
and check to make sure that only the key(s) you wanted were added.
```

```
charles-vence-espinas@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa charles-vence-espinas@server2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/charles-vence-espinas/.ssh/id_rsa.pub"
The authenticity of host 'server2 (192.168.56.106)' can't be established.
ED25519 key fingerprint is SHA256:E9qTtb1dGPl1heGAH5Z1yGPFrZ4cSJLpnErrqjLlFJk.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
    ~/.ssh/known_hosts:5: [hashed name]
    ~/.ssh/known_hosts:6: [hashed name]
    ~/.ssh/known_hosts:7: [hashed name]
    ~/.ssh/known_hosts:8: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
charles-vence-espinas@server2's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'charles-vence-espinas@server2'"
and check to make sure that only the key(s) you wanted were added.
```

3. Once the public key has been configured on the server, the server will allow any connecting user that has the private key to log in. During the login process, the client proves possession of the private key by digitally signing the key exchange.

4. On the local machine, verify that you can SSH with Server 1 and Server 2. What

did you notice? Did the connection ask for a password? If not, why?

```
charles-vence-espinas@workstation:~$ ssh charles-vence-espinas@192.168.56.105
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-28-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Jul 30 21:14:33 2026 from 192.168.56.104
charles-vence-espinas@server1:~$
logout
Connection to 192.168.56.105 closed.
```

```
charles-vence-espinas@workstation:~$ ssh charles-vence-espinas@192.168.56.106
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-28-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Thu Jul 30 21:15:26 2026 from 192.168.56.104
charles-vence-espinas@server2:~$
logout
Connection to 192.168.56.106 closed.
```

-I noticed that I was able to log in to both server 1 and server 2 successfully right after running the SSH command without entering a password. It did not ask for a password because I already copy the public SSH key which is the id_rsa.pub to both servers using the command ssh-copy-id command in the previous steps. When I connected, my local machine proved its identity using its private key which is the id_rsa which it matched the public key which is the id-rsa.pub that is saved in the two servers authorized_keys file.

Part 2: Discussion

Provide screenshots for each task.

It is assumed that you are done with the last activity (Activity 2: SSH Key-Based Authentication).

Set up Git

At the heart of GitHub is an open-source version control system (VCS) called Git. Git is responsible for everything GitHub-related that happens locally on your computer. To use Git on the command line, you'll need to download, install, and configure Git on your computer. You can also install GitHub CLI to use GitHub from the command line. If you

don't need to work with files locally, GitHub lets you complete many Git-related actions directly in the browser, including:

- Creating a repository
- Forking a repository
- Managing files
- Being social

Task 3: Set up the Git Repository

1. On the local machine, verify the version of your git using the command which git. If a directory of git is displayed, then you don't need to install git. Otherwise, to install git, use the following command: sudo apt install git

```
charles-vence-espinas@workstation:~$ sudo apt install git
[sudo] password for charles-vence-espinas:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libwoff1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  git-man liberror-perl
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  git git-man liberror-perl
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 4,806 kB of archives.
After this operation, 24.5 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 liberror-perl all 0.17029-2 [25.6 kB]
Get:2 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 git-man all 1:2.43.0-1ubuntu7.3 [1,100 kB]
Get:3 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 git amd64 1:2.43.0-1ubuntu7.3 [3,680 kB]
Fetched 4,806 kB in 1s (7,277 kB/s)
```

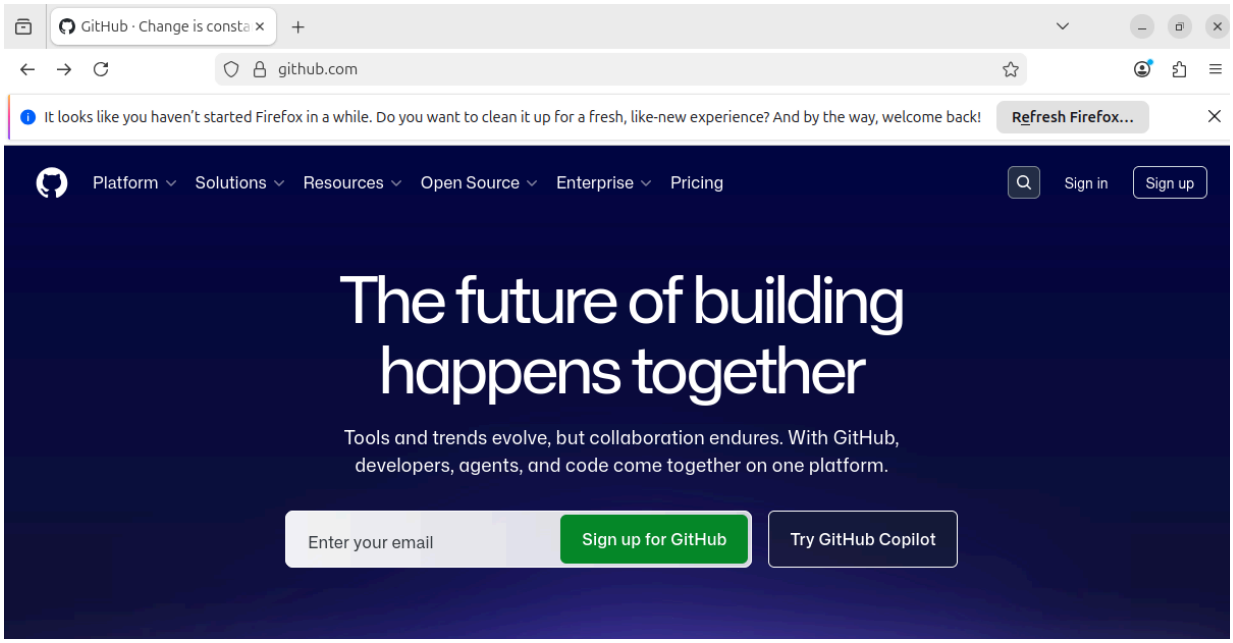
2. After the installation, issue the command which git again. The directory of git is usually installed in this location: user/bin/git.

```
charles-vence-espinas@workstation:~$ which git
/usr/bin/git
```

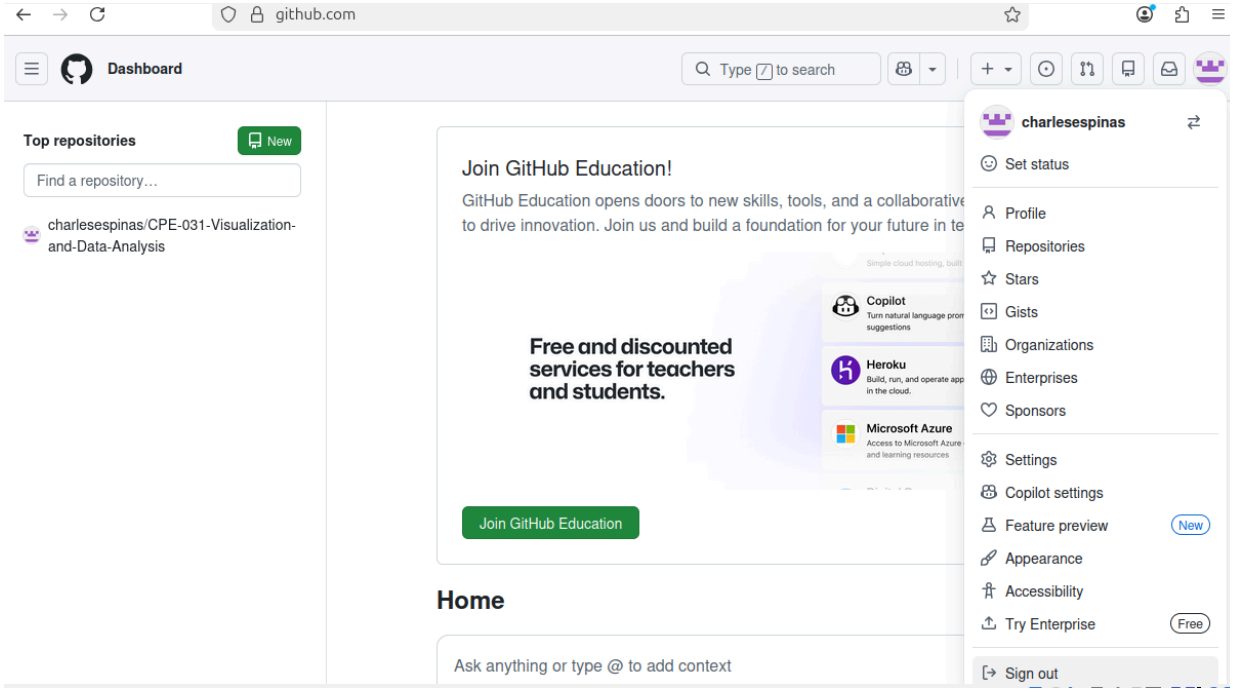
3. The version of git installed in your device is the latest. Try issuing the command git --version to know the version installed.

```
charles-vence-espinas@workstation:~$ git --version
git version 2.43.0
```

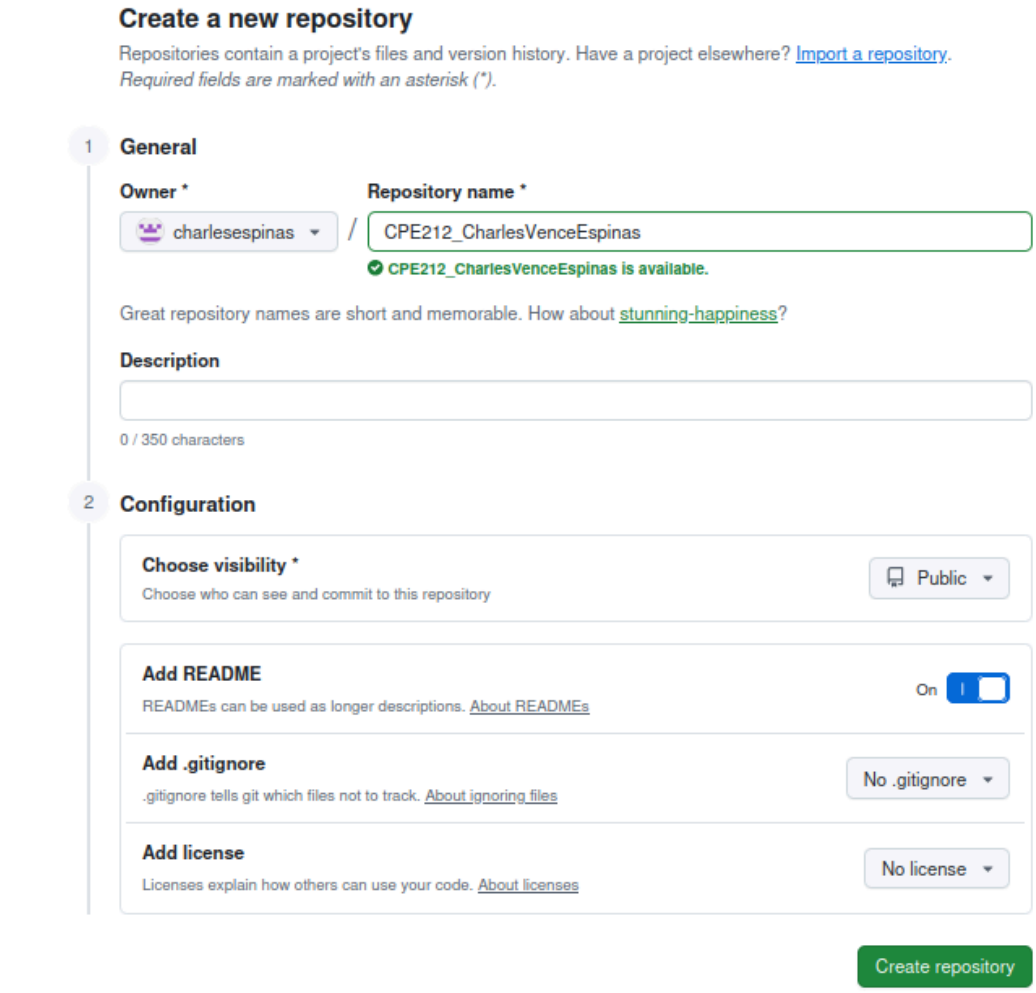
4. Using the browser in the local machine, go to www.github.com.

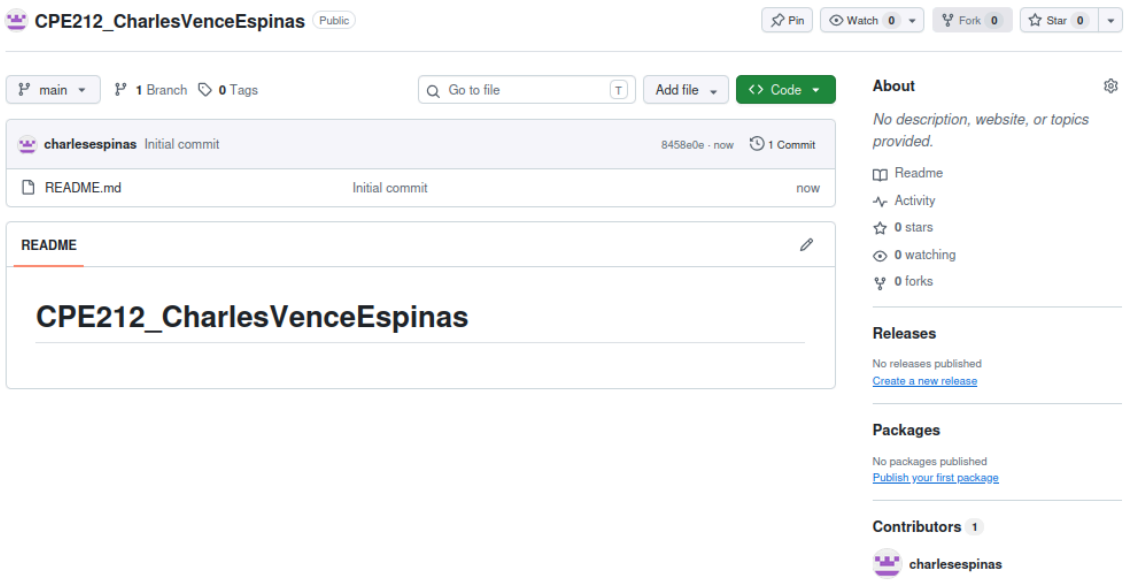


5. Sign up in case you don't have an account yet. Otherwise, login to your GitHub Account.



a. Create a new repository and name it as CPE232_yourname. Check Add a README file and click Create repository.

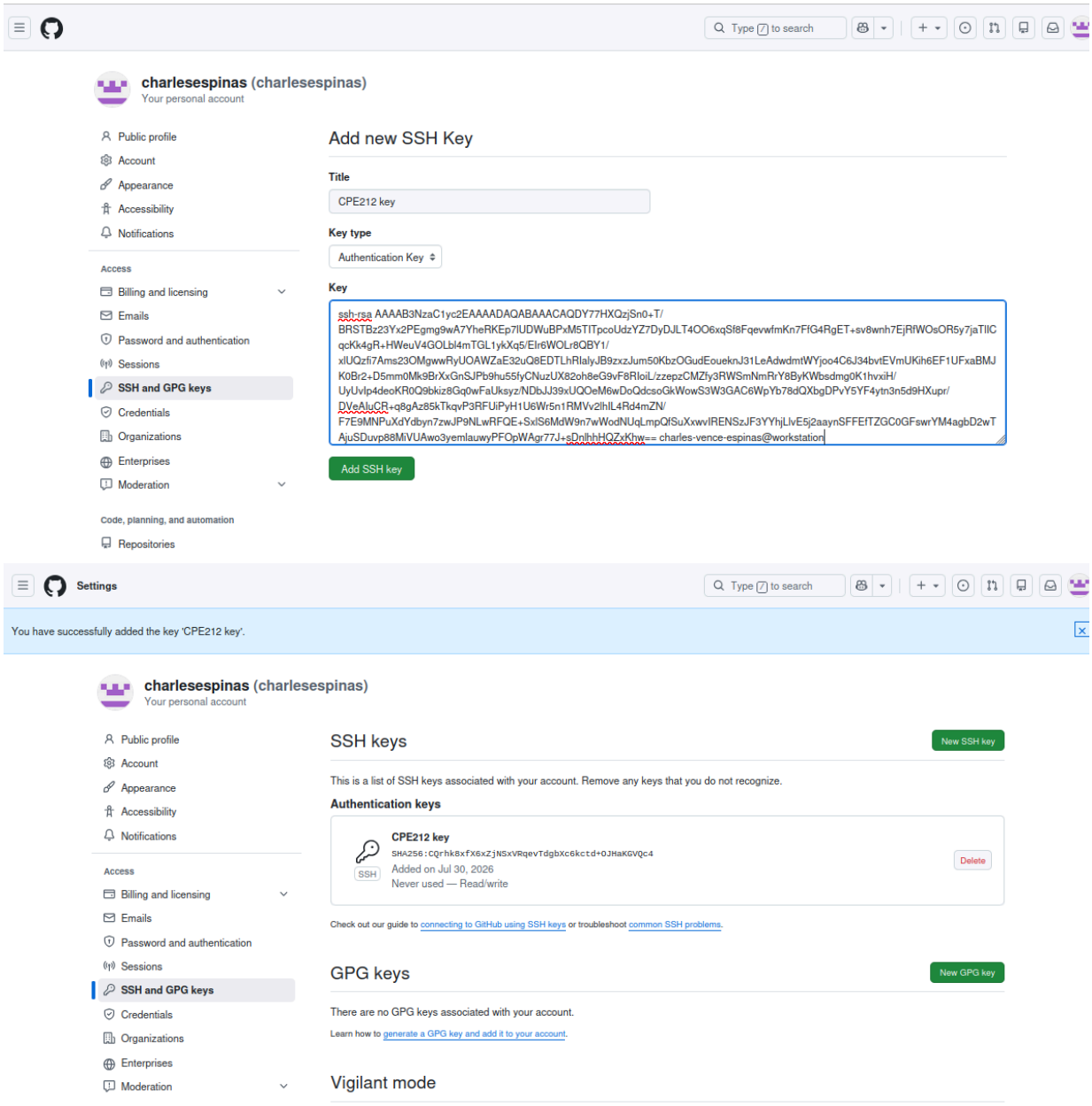

Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).
1 General
Owner *
Repository name *
charlesespinas
/
CPE212_CharlesVenceEspinas
CPE212_CharlesVenceEspinas is available.
Great repository names are short and memorable. How about stunning-happiness?
Description
0 / 350 characters
2 Configuration
Choose visibility *
Choose who can see and commit to this repository
Public
Add README
READMEs can be used as longer descriptions. About READMEs
On
Add .gitignore
.gitignore tells git which files not to track. About ignoring files
No .gitignore
Add license
Licenses explain how others can use your code. About licenses
No license
Create repository



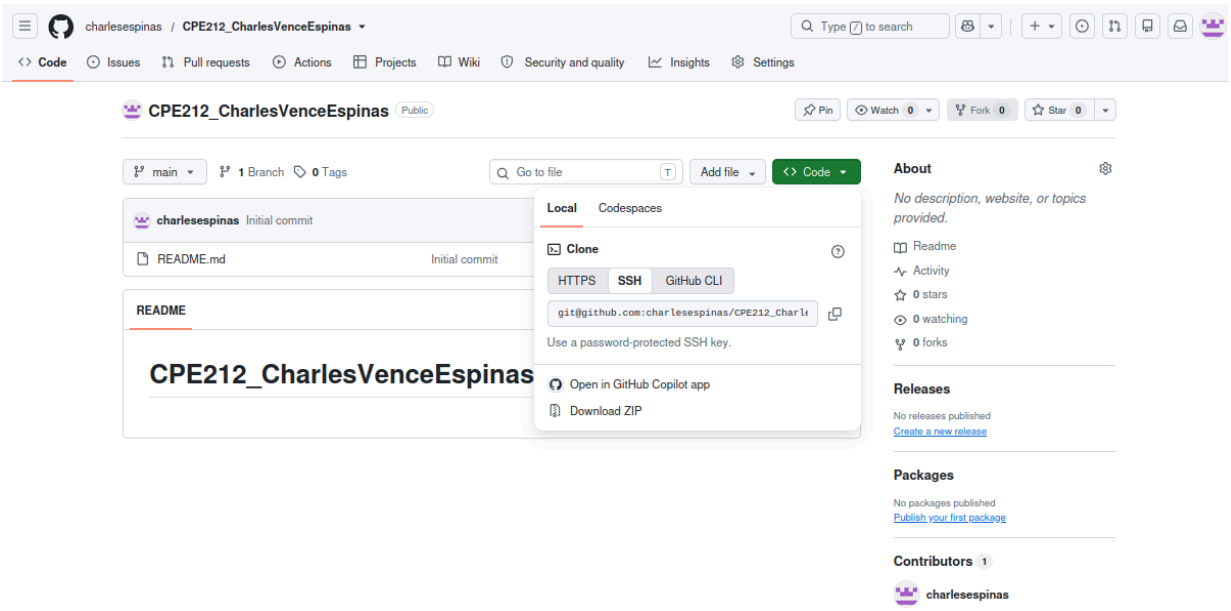
b. Create a new SSH key on GitHub. Go your profile's setting and click SSH and GPG keys. If there is an existing key, make sure to delete it. To create a new SSH keys, click New SSH Key. Write CPE232 key as the title of the key.

c. On the local machine's terminal, issue the command cat .ssh/id_rsa.pub and copy the public key. Paste it on the GitHub key and press Add SSH Key.

```
charles-vence-espinas@workstation:~$ cat .ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQDY77HXQzjSn0+T/BRSTBz23Yx2PEgmg9wA7YheRKEp7lUDWuBPxM5TITpcoUdzYZ7DyDJLT4OO6xq
Sf8FqevwfmKn7FfG4RgET+sv8wnh7EjRfWOsOR5y7jaTlICqcKk4gR+HWeuV4GOLbl4mTGL1ykXq5/EIr6WOLr8QBY1/xlUQzfi7Ams23OMgwwRyUOA
WZaE32uQ8EDTLhRIalyJB9zxzJum50KbzOGudEoueknJ31LeAdwdmtWYjoo4C6J34bvtEVmUKih6EF1UFxaBMJK0Br2+D5mm0Mk9BrXxGnSJPb9hu55
fyCNuzUX82oh8eG9vF8RIoiL/zzepzCMZfy3RWSmNmRrY8ByKWbsdmg0K1hvxiH/UyUvIp4deoKR0Q9bkiz8Gq0wFaUksyz/NDbJJ39xUQOeM6wDoQc
csoGkWowS3W3GAC6WpYb78dQXbgDPvY5YF4ytn3n5d9HXupr/DVeAIuCR+q8gAz85kTkqvP3RFUiPyH1U6Wr5n1RMVv2lhIL4Rd4mZN/F7E9MNPuXdV
dbyn7zwJP9NLwRFQE+SxlS6MdW9n7wWodNUqLmpQfSuXxwvIRENSzJF3YYhjLlvE5j2aaynSFFEfTZGC0GFswrYM4agbD2wTAjuSDuvp88MiVUAwo3y
emlauwyPFOpWAgr77J+sDnlhhHQZxKhw== charles-vence-espinas@workstation
```



d. Clone the repository that you created. In doing this, you need to get the link from GitHub. Browse to your repository as shown below. Click on the Code drop down menu. Select SSH and copy the link.



e. Issue the command git clone followed by the copied link. For example, git clone git@github.com:jvtaylar-cpe/CPE232_yourname.git. When prompted to continue connecting, type yes and press enter.

```
charles-vence-espinas@workstation:~$ git clone git@github.com:charlesespinas/CPE212_CharlesVenceEspinas.git
Cloning into 'CPE212_CharlesVenceEspinas'...
The authenticity of host 'github.com (20.205.243.166)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

f. To verify that you have cloned the GitHub repository, issue the command ls. Observe that you have the CPE232_yourname in the list of your directories. Use CD command to go to that directory and LS command to see the file README.md.

```
charles-vence-espinas@workstation:~$ ls
ansible-project             Desktop    Downloads  Pictures  snap       Videos
CPE212_CharlesVenceEspinas  Documents  Music      Public    Templates
charles-vence-espinas@workstation:~$ cd CPE212_CharlesVenceEspinas
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas$ ls
README.md
```

g. Use the following commands to personalize your git.

- git config --global user.name “Your Name”
- git config --global user.email yourname@email.com
- Verify that you have personalized the config file using the command cat ~/.gitconfig

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas$ git config --global user.name "Charles Vence Espin
s"
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas$ git config --global user.email qcvrespinas@tip.edu
ph
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas$ cat ~/.gitconfig
[user]
        name = Charles Vence Espinas
        email = qcvrespinas@tip.edu.ph
```

h. Edit the README.md file using nano command. Provide any information on the markdown file pertaining to the repository you created. Make sure to write out or save the file and exit.

```
  GNU nano 7.2                                        README.md
#CPE212_CharlesVenceEspinas

## CPE232 - Automating Server Management

Welcome to my repository for ** CPE 212**. This repository contains lab submissions, server management
configurations, and automation scripts developed throughout the course.

---

## Personal  Information
* **Name:** Charles Vence Espinas
* **Course:** CPEP 212 - Automating Server Management

---
```

i. Use the git status command to display the state of the working directory and the staging area. This command shows which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show any information regarding the committed project history. What is the result of issuing this command?

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

j. Use the command git add README.md to add the file into the staging Area.

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas$ git add README.md
```

k. Use the git commit -m "your message" to create a snapshot of the staged changes along the timeline of the Git projects history. The use of this command is required to select the changes that will be staged for the next Commit.

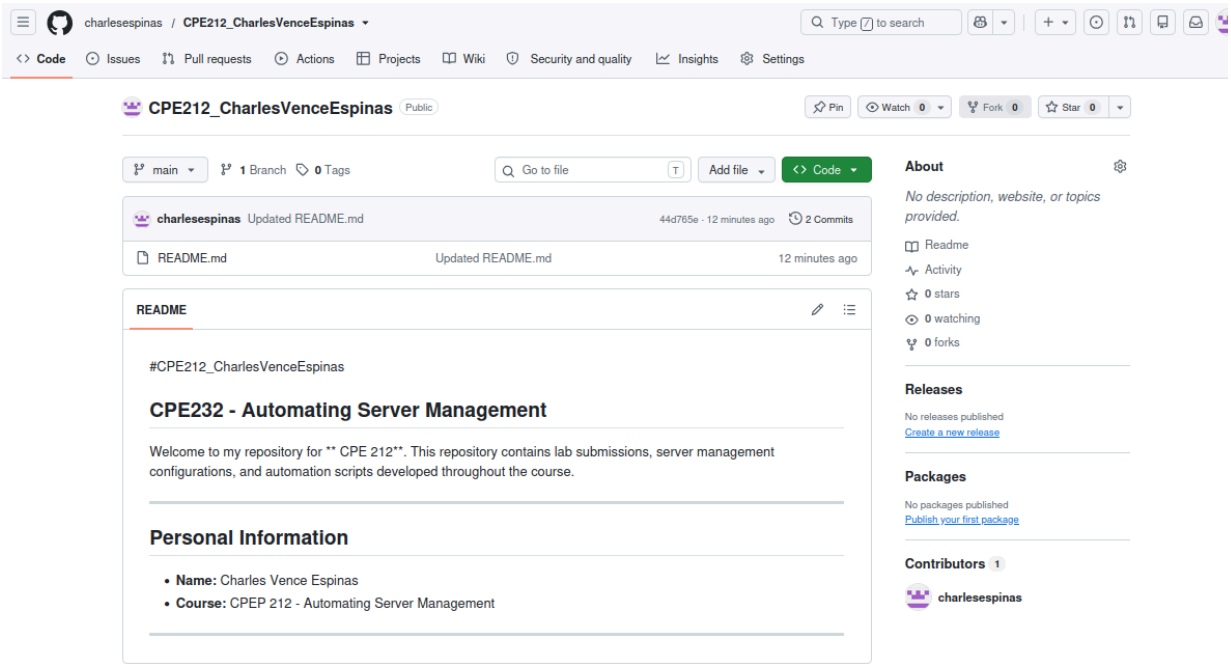
```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas$ git commit -m "Updated README.md"
[main 44d765e] Updated README.md
 1 file changed, 14 insertions(+), 1 deletion(-)
```

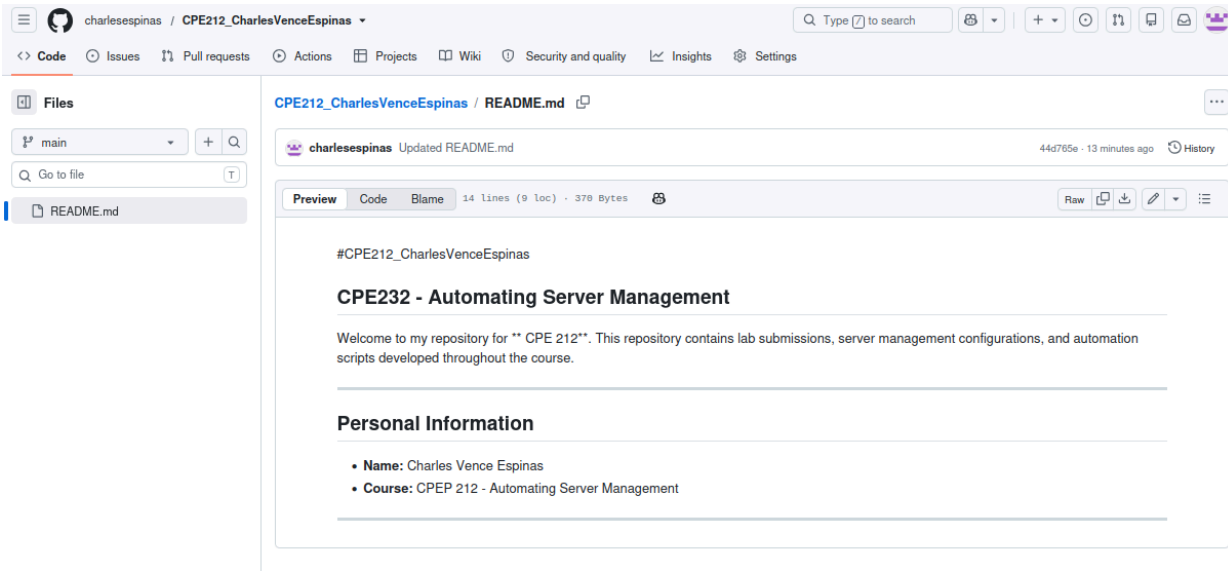
l. Use the command git push <remote><branch> to upload the local repository content to GitHub repository. Pushing means to transfer commits from the local repository to the remote repository. As an

example, you may issue git push origin main.

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 490 bytes | 490.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:charlesespinas/CPE212_CharlesVenceEspinas.git
   8458e0e..44d765e  main -> main
```

m. On the GitHub repository, verify that the changes have been made to README.md by refreshing the page. Describe the README.md file. You can notice the how long was the last commit. It should be some minutes ago and the message you typed on the git commit command should be there. Also, the README.md file should have been edited according to the text you wrote.





2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reflections:

Answer the following:

1. How will you describe the ssh-program? What does it do?
   - The Secure Shell or SSH program is a network protocol and tool that lets us users securely access and control a remote machine over an encrypted connection. This allows us to run terminal commands, manage files and execute tasks on remote servers as if we were sitting right infront of them and at the same time, encrypting the traffic so unauthorized users on the network cannot steal our password or intercept our data.
2. How do you know that you already installed the public key to the remote servers?
   - I will know that I already installed the public key to the remote server when I try to SSH into the remote server and it log me in instantly without asking for a password. Also, I can know through logging in to the remote server and check that the copied key exists inside the file ~/.ssh/authorized_keys.
3. What sort of things have we so far done to the remote servers using ansible commands?
   - Nothing yet.
4. How important is the inventory file?
   - The inventory file is very important because it serves as a central directory or roadmap for the Ansible. It defines the IP addresses, hostnames and groups of all remote target servers that ansible manages. Without an inventory file, Ansible will not know which remote machines to connect or where to execute the commands and playbooks.

Conclusions/Learnings:

This activity helped me to understand how the SSH key authentication provides a secure and a convenient way to access the remote servers without entering a password for each server everytime. I also learned and also enhanced my knowledge how to generate the SSH keys, copy the public key to the remote servers and verify if the authentication are working correctly. Also, I experience setting up Git, creating a GitHub repository, cloning it, tracking file changes and pushing commits to a remote repository. To be honest, I only know how to create a repository since it is very easy but because of this hands on activity, I gained a good and amazing experience for the first time on setting up the Git and GitHub in the command line interface in the ubuntu. Moreover, these tasks

showed me how SSH and Git work together to make remote server management and version control more secure and efficient. Overall, this activity improved my knowledge and skill in using Linux and SSH commands, Git and GitHub which are very important in my future academic and career.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.